

GAME DEVELOPMENT JOURNEY REPORT: BUILDING A CUTE & FEATURE-RICH SNAKE GAME

Date: June 19, 2026 | *Author:* Game Developer & AI Assistant Collaboration

1. Executive Summary

This report documents the iterative software development lifecycle of a custom JavaScript-based Snake game. Developed collaboratively using AI models (Gemini and ChatGPT), the game evolved from a standard, functional arcade game ("Neon Overdrive Snake") into a highly personalized, addictive, and visually charming application ("Cute Snake"). The journey highlights a significant learning curve in prompt engineering, debugging code crashes, and managing state synchronization across multiple feature sets.

2. Prompt Iterations & Evolution Over Time

The development process was driven entirely by iterative prompting. Over time, the strategy shifted from broad feature requests to highly constrained, targeted prompts to avoid breaking legacy code.

Development Phase	Prompt Style Used	AI Model & Strategy
Phase 1: Core Engine	Basic feature description (Grid mechanics, basic snake movement, food spawning).	Leveraged standard structural templates. Focus was entirely on functional correctness.
Phase 2: Mechanics Expansion	Adding high scores, random map hurdles, particle effects on food consumption, and diamond currency.	Alternated between models to refine math logic and particle physics array slicing.
Phase 3: Advanced Logic (Power-Ups)	Structured, variable-focused prompt defining local state timers, speed intervals (Speed Boost & Slow-Mo).	Introduced strict constraints preventing modification of existing data stores.
Phase 4: Aesthetic Overhaul	Visual modification prompt substituting standard rendering functions with rounded shapes and pastel colors.	Explicit layout definitions to transform sharp neon blocks into soft canvas bubbles and emojis.

How Prompts Improved Over Time

- Early Stage:** Prompts were open-ended (e.g., "Add power-ups to my game"). This frequently resulted in the AI rewriting the entire code block and accidentally dropping existing features.
- Late Stage:** Prompts became highly algorithmic and constraint-heavy. For example, explicitly telling the AI: *"Constraint: Mere purane logic (High score, hurdles, particles, diamonds) mein koi badlav nahi aana chahiye."*

Code sirf Power-Up ka logic add karne ke liye modify karein." This structured engineering approach drastically minimized regressions.

3. What Worked Well

- **Modular Phase Transitions:** Breaking down features sequentially (Score System -> Currency -> Obstacles -> Power-Ups -> Aesthetics) allowed for isolated testing before full deployment.
- **Dynamic SetTimeout Interactivity:** Adjusting the game speed dynamically via a global `currentSpeed` variable in the `gameLoop` seamlessly supported smooth Speed Boosts (60ms) and Slow-Mo (250ms) modes.
- **State Persistence:** Utilizing the browser's `localStorage` API for both high scores and diamond currency worked flawlessly across reboots.
- **Cross-Model Verification:** Utilizing both ChatGPT and Gemini provided a hybrid advantage—one model excelled at drafting rapid visual prototypes, while the other provided superior logic debugging when complex arrays broke.

4. What Did Not Work & Major Code Crashes

Developing with AI without absolute constraints led to several critical system breakdowns:

1. **The Draw Loop Collision (The Visual Overwrite Crash):** During the transition from a Neon layout to a Cute/Bubbly theme, the request to replace `fillRect()` with rounded shapes caused the AI to completely wipe out the rendering arrays for hurdles, currency collectibles, and custom food particles. The game executed without errors, but the map elements were entirely missing visually.
2. **Speed Reset Failure:** In early iterations of the Power-Up subsystem, once a Speed or Slow-Mo buff timed out, the loop interval failed to restore to the base speed (125ms), locking the snake in permanent warp or freeze frames.
3. **State Inconsistency Across Refactors:** When copying code back and forth between different AI instances, variable initializations (like `lastScoreMilestone`) were occasionally dropped, causing infinite hurdles to spawn simultaneously.

5. Challenges Faced & Key Learnings

Challenges:

- Managing the context limit of AI models when pasting 200+ lines of raw HTML/JS.
- Ensuring HTML5 Canvas layers render objects in the correct order (e.g., rendering particles above the background but behind the text overlays).

Key Learnings for Future AI-Assisted Coding:

- **Always Supply Complete Current State:** Never ask an AI to modify a game based on a memory of a past session. Always paste the latest, fully operational code.
- **Enforce Rigid Code Isolation:** Instruct the AI precisely which functions to modify (e.g., *"Only modify the draw() block"*) to avoid random omissions elsewhere.

- **Incremental Commits:** Keep backup versions of your working code (such as index files) before pasting experimental prompts.

6. Conclusion

The journey from a basic neon configuration to a fully synchronized, bubble-themed, feature-rich game demonstrates the true power of collaborative development. By mastering prompt refinement, understanding structural constraints, and turning code failures into architectural insights, a robust web application was successfully launched and prepared for cloud deployment on GitHub Pages.